Aran Learn 13

Aran Learn 13: Animation and Visual Explainers

For: everyone, before touching the animated tour. Inbathamizhan owns the tour shell; Bharadhwaj and Arun own the detector, gateway and immune scenes; Aasif owns the token, leak-trace and erasure scenes. **Time:** about 4 hours with practice. **Before this:** Learn 11 (HTML, CSS, React, `useState`, `useEffect`).

The guided tour now explains Aran with moving pictures instead of paragraphs: packets flow through the firewall, a token is built round by round, check-digit maths runs digit by digit, a key shatters on erasure. This doc teaches the four tools behind every one of those scenes: **SVG**, **CSS transitions**, **CSS keyframe animations**, and **step-by-step animation driven by React state**. It ends with the one rule that matters most: every animation must replay real data from the server, never a made-up example.

1. SVG: drawing with code

SVG (Scalable Vector Graphics) is HTML for shapes. It lives inside the page like any element, so CSS and React can change it.

```
        <svg viewBox="0 0 400 120" width="100%">
          <rect x="10" y="40" width="90" height="40" rx="8" fill="var(--surface-2)"
stroke="var(--border)" />
          <text x="55" y="65" text-anchor="middle" font-size="13">Loan app</text>
          <line x1="100" y1="60" x2="300" y2="60" stroke="var(--border)" stroke-dasharray="4
4" />
          <circle cx="200" cy="60" r="6" fill="var(--leak)" />
        </svg>
```

- **`viewBox="0 0 400 120"`** defines the drawing's own coordinate system: 400 units wide, 120 tall. `width="100%"` then scales the whole picture to fit the screen. This is why the tour's diagrams work on a phone without extra code.
- **Shapes:** `rect` (box, `rx` rounds corners), `circle` (`cx`, `cy`, `r`), `line`, `path` (any curve), `text`, and `g` (a group you can move as one).
- **Colours come from our tokens** (`var(--leak)`, `var(--token)`), so light and dark themes work for free. Never write a hex colour inside a scene.
- **Paths.** `M 10 60 C 150 0, 250 120, 390 60` means "move to (10,60), then draw a curve to (390,60) bent by two control points". The tour's packet lanes are paths like this.

2. CSS transitions: smooth changes

A transition animates a property whenever its value changes:

```
        .cell { background: var(--surface-2); transition: background .4s ease-out, transform
.4s ease-out; }
        .cell.hot { background: var(--leak-bg); transform: scale(1.15); }
```

Add the class `hot` with React and the cell fades and grows; remove it and it shrinks back. Good for: highlighting the digit being processed, a bar filling to a probability, a token cell turning amber.

Only animate `transform` and `opacity` when you can. They are cheap for the browser; animating `width` or `top` makes the page stutter on slow laptops.

3. CSS keyframes: animations that run on their own

```
        @keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: .35; } }
        .live-dot { animation: pulse 1.6s infinite; }
```

To move a shape along a path inside SVG, use SVG's own `<animateMotion>`; it works in every browser and uses the drawing's coordinates:

```
        <circle r="6" fill="var(--leak)">
          <animateMotion dur="2.4s" fill="freeze" path="M 10 60 C 150 0, 250 120, 390 60" />
        </circle>
```

- `@keyframes` names a sequence of states; `animation:` plays it with a duration, an easing and a repeat count.
- `<animateMotion>` moves a shape along a path. The tour uses it for packets travelling from a source system through Aran to a sink.
- `forwards` (CSS) or `fill="freeze"` (SVG) keeps the last frame when the animation ends. `infinite` loops.
- To restart a keyframe animation from React, change the element's `key`: React throws the old element away and the animation plays again from the start.

4. Step-by-step animation with React state

Most tour scenes are not one smooth movement; they are a story told in steps: "digit 12 → digit 11 → … → result". The pattern is a counter in state and a timer that advances it:

```
        function useStepper(total, ms = 450) {
          const [i, setI] = useState(0)
          const [playing, setPlaying] = useState(false)
          useEffect(() => {
            if (!playing) return
            if (i >= total) { setPlaying(false); return }
            const t = setTimeout(() => setI(i + 1), ms)
            return () => clearTimeout(t)          // stop the timer if the page changes
          }, [playing, i, total, ms])
          return { i, playing, play: () => { setI(0); setPlaying(true) }, step: () =>
```

```
setI(Math.min(i + 1, total)) }
        }
```

Then the scene simply draws “everything up to step `i`”:

```
        const { i, play } = useStepper(steps.length)
        {steps.map((s, n) => <span key={n} className={n < i ? 'done' : n === i ? 'now' :
''}>{s.digit}</span>)}
```

Every animated scene in `app/web/src/viz.jsx` uses this hook, so each one has the same Play, Step and Replay buttons and a person presenting to a reviewer can pause on any frame and explain it.

5. Counting up and drawing charts

- **Count-up numbers.** A number that climbs from 0 to 705 in a second draws the eye far more than a static 705. `CountUp` in `viz.jsx` uses `requestAnimationFrame`, the browser’s “call me before the next screen refresh” timer, to update the number about 60 times a second.
- **Radar charts.** The immune scene draws seven spokes, one per feature the immune layer measures. A traffic window is a polygon: one point per spoke at the feature’s value. The detector scales every feature so that what it saw during learning sits between 0.1 and 0.7, so the green ring between those radii is literally “normal” and anything poking outside it is unusual.

6. Accessibility and motion

- Some people get dizzy from motion. The stylesheet already turns animations off when the computer’s “reduce motion” setting is on (`@media (prefers-reduced-motion: reduce)`). Step scenes still work: they just jump between frames.
- Never rely on motion or colour alone. A caught value also gets a ✓ and a word; a leaked one gets “raw”.
- Every SVG gets `role="img"` and an `aria-label` that says in words what the picture shows.

7. The one rule: animate real data

A beautiful animation of a fake example is worse than none: a reviewer who later finds the numbers were invented will trust nothing else on the page. In Aran every scene is a replay of a real server response:

Scene	What it replays	Endpoint
Packets through the firewall	a real sample request through the live firewall	`POST /api/tour/packet`
Building a token	the ten real FF1 rounds, plus a check that the firewall issued the same token	`POST /api/tour/build-token`

Scene	What it replays	Endpoint
Check-digit maths	the server's own Verhoeff or Luhn steps	`POST /api/tour/checksum`
False-alarm rain	300 real log lines, one cell each	`POST /api/tour/false-alarms`
Leak trace	the trace of a real leaked sink's text	`POST /api/trace`
Key shatter and receipt	a real erasure, its Merkle proof and signature check	`POST /api/erase`, `/api/receipts/verify`
Immune radar	the detector's own scaled features for recent windows	`GET /api/tour/immune-radar`
AI gateway	the real gateway on the live detector	`POST /api/tour/gateway`

If a scene needs data the server does not give yet, add it to the server first (see `app/server/engine.py`, functions starting `tour_`), then animate it.

Practice

1. **Draw.** In a blank HTML file, draw two boxes joined by a dashed line, with a small circle on the line. Make it scale to the window width.
2. **Move.** Give the circle an `<animateMotion>` along your line lasting 2 seconds. Make it loop.
3. **Step.** Copy `useStepper` and build a row of 12 digit cells that light up one by one from right to left when you press Play.
4. **Replay real data.** Call `POST /api/tour/checksum` with a generated Aadhaar from `GET /api/tour/generate` and make your stepper walk through `steps`, showing `before → after` for each digit.
5. **Reduce motion.** Turn on "reduce motion" in your operating system and check your page still makes sense.

Solutions

1. `<svg viewBox="0 0 400 80" width="100%"><rect x="10" y="20" width="80" height="40" rx="6" fill="var(--surface-2)"/><rect x="310" y="20" width="80" height="40" rx="6" fill="var(--surface-2)"/><line x1="90" y1="40" x2="310" y2="40" stroke="gray" stroke-dasharray="4 4"/> <circle class="dot" r="6" fill="crimson"/></svg>`
2. Inside the circle: `<animateMotion dur="2s" repeatCount="indefinite" path="M 90 40 L 310 40" />`.
3. `const { i, play } = useStepper(12)` and `digits.split('').reverse().map((d, n) => <span className={n < i ? 'done' : ''}>{d}</span>)`.
4. `const r = await api('/tour/checksum', { digits: (await api('/tour/generate')).aadhaar })`, then render `r.steps.slice(0, i)`.

5. With reduced motion, `useStepper` still advances, so cells change instantly instead of fading; nothing is lost.